

P2513R2

char8_t Compatibility and Portability Fix

Published Proposal, 2022-05-15

Authors:

[JeanHeyd Meneide \(https://theohd.dev\)](https://theohd.dev)

[Tom Honermann](#)

Audience:

CWG

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Target:

C++26 / C++23 (DR C++20)

Latest:

https://theohd.dev/_vendor/future_cxx/papers/d2513.html

Abstract

char8_t has compatibility problems and issues during deployment that people have had to spend energy working around. This paper aims to alleviate some of those compatibility problems, for both C and C++, around string and character literals for the char8_t type.

Table of Contents

1	Revision History
1.1	Revision 2 - May 15th, 2022
1.2	Revision 1 - February 15th, 2022
1.3	Revision 0 - January 15th, 2022
2	Polls & Votes
2.1	May 12th, 2022 - EWG
2.2	February 9th, 2022 - SG16
3	Introduction and Motivation
3.1	C Compatibility
3.2	Compatibility Troubles in Existing Libraries
4	Design
4.1	Why unsigned char?
4.2	Casting/aliasing?
4.3	C Compatibility
4.4	Defect Report
4.5	What about special unsigned char* rules?
4.5.1	Compound Literals with C?
4.5.2	But you CAN make it work??
4.6	Overload Resolution for Array-Containing Structure Initialization
5	Specification
5.1	Language Wording

- 5.1.1 Adjust Feature Test Macro for `char8_t` in [tab:cpp.predefined.ft]
- 5.1.2 Modify Initialization of Character Arrays in [dcl.init.string]
- 5.1.3 Add Annex C.1.6 example for change in code [diff.cpp20]

References

Informative References

§ 1. Revision History

§ 1.1. Revision 2 - May 15th, 2022

- Add new Tony Table.
- Passed EWG with the addition of excluding `signed char` explicitly from the `u8` initialization rules.
- Wording updated to reflect this behavior.

§ 1.2. Revision 1 - February 15th, 2022

- Fix typos and other grammar mistakes in various sections such as in [§ 4.2 Casting/Aliasing?](#).
- Use "may" in both places in the wording, rather than "can" and then "may".
- "Fix" for the title, rather than "Fixes".
- Discuss the aggregate-initialization-with-overloading case related to fixed-size arrays and brace initialization in [§ 4.6 Overload Resolution for Array-Containing Structure Initialization](#).
- Adjust wording to include Annex C entry in [§ 5.1.3 Add Annex C.1.6 example for change in code \[diff.cpp20\]](#).
- Successfully passed SG16 vote to be forwarded to EWG, potentially for C++23.

§ 1.3. Revision 0 - January 15th, 2022

- Initial Release! 🎉

§ 2. Polls & Votes

Votes are done in a Strongly in Favor (SF) / Favor (F) / Neutral (N) / Against (A) / Strongly Against (SA) format. Differences between vote count and number of attendees is abstention.

§ 2.1. May 12th, 2022 - EWG

Accept P2513R1 as a Defect Report against C++20.

SF F N A SA 3 5 2 1 0

Result: Consensus (8-1)

Accept P2513R1, with the modification to exclude 'signed char' from the allowable conversions list as a Defect > > Report against C++20.

SF F N A SA 5 4 1 1 0

Result: Consensus (9-1) < Stronger

The second poll has stronger consensus, so it will be forwarded to electronic polling.

The one to remove `signed char` had slightly higher consensus, so it was chosen since the authors had no preference.

Add an Annex C entry and discussion to D2513R1, and forward the published paper as revised to EWG as a defect report.

SF F N A S A

1 5 0 1 0

Attendance: 8

Author position: SF

Consensus: Strong consensus

Against rationale: Adding another weird inconsistency between pointers and arrays; discussion decreased comfort; breakage is concerning.

§ 3. Introduction and Motivation

Pre-C++20	<pre>const char* ptr0 = u8""; // Works ✓ const unsigned char* ptr1 = u8""; // Does not work ✗ const char arr0[] = u8""; // Works ✓ const unsigned char arr1[] = u8""; // Works ✓</pre>	<pre>constexpr const char* resource_i return u8"o(*∇*)o"; / }</pre>
C++20	<pre>const char* ptr0 = u8""; // Broken ♥ const unsigned char* ptr1 = u8""; // Still does not work ✗ const char arr0[] = u8""; // Broken ♥ const unsigned char arr1[] = u8""; // Broken ♥</pre>	<pre>constexpr const char* resource_i return u8"o(*∇*)o"; / }</pre>
C++20- with- DR	<pre>const char* ptr0 = u8""; // Still Broken ♥ const unsigned char* ptr1 = u8""; // Still does not work ✗ const char arr0[] = u8""; // Works Again ✨✓ const unsigned char arr1[] = u8""; // Works Again ✨✓</pre>	<pre>constexpr const char* resource_i const char res_id[] = u8 return res_id; // Works }</pre>

The introduction of `char8_t` has introduced backwards **and** forward compatibility issues into the C++ ecosystem, and also issues with C compatibility as well. Despite Tom Honermann's [P1423r3], the direct incompatibility between `char` and `char8_t` was felt, enough that `-fno-char8_t` and `/Zc:char8_t-` needed to be rolled out the moment conforming C++20-aspiring implementations rolled out with the `char8_t` changes to prevent breakages. (For `-fno-char8_t`, it was implemented when `char8_t` was rolled out. For `/Zc:char8_t-`, it was implemented after a beta testing period under users with `/std:c++latest` that [resulted in a handful of projects reporting broken codebases, such as dear imgui.](#))

Among the breakages, ones that stood out were that several kinds of string initialization and pointer conversions were illegal, particular ones involving `char`:

```
const char* a = u8"a"; // broken in C++20
const char b[] = u8"b"; // broken in C++20
const unsigned char c[] = u8"c"; // broken in C++20
```

This has also exasperated `constexpr` concerns, where it is fundamentally impossible to convert between types with a `reinterpret_cast` and therefore requires a special "shim" layer to copy elements from one array type to another:

```
#include <utility>

template<std::size_t N>
struct char8_t_string_literal {
    static constexpr inline std::size_t size = N;
```

```

template<std::size_t... I>
constexpr char8_t_string_literal(const char8_t (&r)[N], std::index_sequence<I...>)
: s{r[I]...} {}

constexpr char8_t_string_literal(
const char8_t (&r)[N])
: char8_t_string_literal(r, std::make_index_sequence<N>()) {}

auto operator <=>(const char8_t_string_literal&) = default;

char8_t s[N];
};

template<char8_t_string_literal L, std::size_t... I>
constexpr inline const char as_char_buffer[sizeof...(I)] =
{ static_cast<char>(L.s[I])... };

template<char8_t_string_literal L, std::size_t... I>
constexpr auto& make_as_char_buffer(std::index_sequence<I...>) {
return as_char_buffer<L, I...>;
}

constexpr char operator ""_as_char(char8_t c) {
return c;
}

template<char8_t_string_literal L>
constexpr auto& operator""_as_char() {
return make_as_char_buffer<L>(std::make_index_sequence<decltype(L)::size>());
}

#if defined(__cpp_char8_t)
#   define U8(x) u8##x##_as_char
#else
#   define U8(x) u8##x
#endif

int main () {
constexpr const char* p = U8("text");
constexpr const char& r = U8('x');
return 0;
}

```

With all due respect to the effort involved, these are solutions only a C++ expert could love. It harkens back to days long-gone-by of `TCHAR` type and `TEXT(...)` macros when programming on Microsoft Windows, which has been regarded with some small amount of disdain in Windows Programming for well over a decade now. It is troublesome to program in this form and communicating to programs not familiar with the convention results in higher operational overhead for developers that need to get used to this. There's also just the risk of forgetting to do this and suffering compile-time breaks that only manifest in certain testing modes (e.g., developing in C++14/17 mode but running Continuous Integration against C++20). This is why the ANSI and Codepage-based functions are discouraged for new applications, and Windows API users are encouraged to use the Unicode-based, `W`-suffixed functions and nothing else. Even non-Microsoft sources encourage this, e.g. explicitly on the [UTF-8 Everywhere Page](#) and Microsoft itself has embraced UTF-8 by instructing application developers to deploy manifests with their program to request UTF-8 data where applicable.

There are other solutions as well, such as constructing a `char_array<N>` type that holds the data. This is a little bit more elegant and usable, but still requires substituting places of character arrays with different types entirely and relying on (implicit) conversions to make it work as expected. This does not play nice with templated functions in C++, and is just completely impossible in C code.

§ 3.1. C Compatibility

Worse, this code impacts C Compatibility both before and after any changes to `u8` or introductions to `char8_t` in the C language. What used to be portable C and C++ code that could live in headers now breaks, similar to the C++17 to C++20 transition:

```
extern const char* a = u8"a"; // Works in C (using default extensions), broken in C++20
extern const char b[] = u8"b"; // Works in C, broken in C++20
extern const unsigned char* c = u8"c"; // Works in C (using default extensions), broken in
extern const unsigned char d[] = u8"d"; // Works in C, broken in C++20
```

This kind of break in previously working code may be too far reaching. Even if the [char8_t for C paper, N2653](#) passes for C23 (or later), it only introduces `char8_t` in a C-style. That is, `char8_t` is simply a type definition for `unsigned char`, similar to how `char16_t` and `char32_t` are defined in library headers for C using `uint_least(16/32)_t`. This still gives us the benefit of type-generic programming in C with `_Generic`, macros, and more, but still leaves us with the compatibility problem. Namely, a construct that should definitely work between C and C++ that break are:

```
extern const unsigned char d[] = u8"d"; // Works in C even after N2653, breaks in C++20
```

These breaks have caused issues, including for very popular C and C++ libraries, and the solution is adding C++20-specific overloads. But this does nothing to help individuals who are trying to write C++11, 14, and 17 code that needs to eventually transition to use `char8_t`. To ease portability between the two languages in shared header code and to enable the ability for individuals porting C++11-to-C++17 code to C++20, this proposal works to allow initialization of `unsigned char` arrays (and other *ordinary character* array types) from `u8` string literals.

§ 3.2. Compatibility Troubles in Existing Libraries

There are many libraries that have sustained usability decreases from the introduction of `const char8_t[]` as the type for `u8` string literals. Popular user libraries such as Dear ImGui, `nlohmann::json`, and many others suffer from these issues. For example:

Basically dear ImGui wants to use low-level types here `const char*` + promote terse code, `u8` was perfect for encoding strings. When using the lib users typically use LOTS of literals. Now users can't without a cast or us adding overloads to several hundreds entry points.

Those users, the majority are silent in the first place, they are used to that kind of software not working well for their languages, they move on. Dear ImGui supported them somehow (very imperfectly but enough to attract a crowd). Now things became much less attractive.

... The lib is designed for very fast iteration, compact code, imho it is a great loss.

— [Omar, Discussion of ImGui, January 4th, 2022](#)

This kind of pain has been repeated in other libraries, such as `nlohmann::json`:

Watch on this! `std::u8string_view` is serialized as number array now.. I have to explicitly convert it into `std::string_view` every time.

— [shrinktofit, Issue #2097, September 28th, 2020](#)

You are right, `std::u8string` is currently not supported. I currently see no blocker in supporting it, but I cannot promise any timeline for the feature. Any help (and PRs) welcome!

— [nlohmann, Issue #1914, January 28th, 2020](#)

The tests for `nlohmann::json` were simply stripped of all their uses of `u8` strings. Where necessary the library (and many others) simply use by-hand byte sequence encoding in non-prefixed string literals when they know they cannot influence the use of command line arguments for UTF-8 encoded strings.

Some code just remains broken currently, such as the [antlr4](#) project which generates `std::strings` using `u8"..."` literals. That will require greater surgery to fix.

This proposal allows for a dedicated migration path, albeit it still require minor changes. In particular, users will have to first create a variable so that the UTF-8 string literal can be used to initialize a `const char[]`, `const unsigned char[]`, or `const char8_t[]` array. Then, the array can be used as expected with the desired type the end-user requires.

§ 4. Design

There are three core goals this proposal is out to achieve, **specifically** around the usage of single `char8_t` literal and `u8` string literals:

- code written in both C and C++ in a header file will initialize and work properly when using `unsigned char`, especially as a migration `typedef` to go to different places;
- code written to be compatible with both pre-C++17 and C++20-and-beyond, as well as C, can work properly by using `unsigned char` to indicate an unsigned at-least-8-bit code unit;
- code that wants to remain compatible with old `u8"..."` literal behavior can initialize to `const char[]` variables or `const signed char[]` variables;
- and, enabling a gradual migration path that is not a hard break that can be mechanically accounted for, rather than requiring larger, more involved and architected changes.

This proposal is the **smallest, simplest possible fix**. It explicitly does not attempt to deal with conversion or use as a pointer value, and deals strictly with array initialization. This means that function calls and initialization of a `const unsigned char*` or `const char*` pointer is not included in this proposal: a future proposal that is a Defect Report may aid in improving usability if a cast-based solution, discussed further below in brief, does not emerge in the C++26 standardization timeframe.

§ 4.1. Why `unsigned char`?

`unsigned char` is the best candidate for a permanent transition path for C++. It will enable people to write code that has the exact same behaviors and semantics as `char8_t`, and transition more seamlessly when support for `char8_t` strings, string literals, character literals, and more is phased into `std::format`, `std::print`, and the standard library.

There is strong in-the-industry usage of `unsigned char` to represent a single UTF-8 code unit, so much so that it has even shown up in [papers from as far back as 2006](#) and also mentioned briefly in [a paper from 2007 \(Appendix, Item #13\)](#) with regards to defining `char8_t` types themselves for their own libraries. It is also a common technique in mature codebases to define `typedef std::basic_string<unsigned char, my_u8_traits> u8string`; as a means to semantically differentiate between a string with potentially any kind of data (or execution encoding data) and UTF-8 data. This is typically the way to handle this in the cases where the programmer is not part of one of the hundred-million, billion, and multi-billion dollar service companies that control their entire computer stacks.

Groups with the power to control the entire vertical stack — from their data centers to the final services running in the browser and on end-user machines — can guarantee that they can simply set their locale to be UTF-8 on their native machine. This is not exactly possible across all tech stacks, however: Microsoft has only [just started to encourage UTF-8, after all](#). However, the option for turning on UTF-8 as the default Active Code Page (ACP) is still hidden in the legacy control panel settings behind 3 dialog boxes and a checkmark to turn on a "BETA" feature. This means that the wide variety of software that still uses `fopen`, command line arguments, `std::fstream`, and more without conversion subject themselves to whatever the execution encoding may happen to be on their machines. For Microsoft software, that is broken just from using the file APIs. On Linux software, even if the file APIs are pass-through, code is broken by way of consuming `const char[]` data in execution encoding and interfacing with file system and other tokens which may not have been stored in that fashion.

Therefore, this proposal focuses on `unsigned char` as a good candidate for a permanent transition path for older-than-C++20 code. Note that this technique has been already deployed to great use in the industry. It was presented on as a "bridging" technique for pre-C++20 code looking for a compile-time way to differentiate their strings and string literals in C++, especially since `std::byte*` can serve as the proper "byte transportation" type:

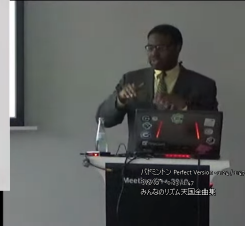
Meeting C++ 2019

JeanHeyd Meneide

Catching up:
Unicode for C++
in Greater Detail
2 of 5

unsigned char: savior before C++20

- Allows you to get hard compiler errors, early
 - using u8string = std::basic_string<unsigned char>;
- 🌸 Blossom 🌸 changes out to program boundaries
 - Explicitly encode/decode or mark when going from/to std::string



(Timed Video Link Describing the Process)

Tapping into this current industry best-practice is a good way to give people in pre-C++20 code practice for working with a `char8_t` world, and provide them a direct migration path if they **do** define their own `my_char8_t` type for use in their codebases, as many companies both old and new have been doing. One such customer used `unsigned char` to eliminate all of the transcoding bugs in their PDF-adjacent plugin software when they began to make that software available outside of Germany, and the technique has been so good that there were no bugs in the entire tech stack once they finished adding all the explicit conversions between `std::string` and their internally-defined `u8string` type using `unsigned char` and a hand-customized `char_traits`. The authors of this proposal also use exactly the same technique in many of the codebases they have been in since before C++20, to great success at drastically reducing encoding bugs.

§ 4.2. Casting/Aliasing?

We do not provide a way for a `char8_t` pointer to be cast into a `char` or `unsigned char` pointer. This would violate type-based alias analysis and the rules for `char8_t`: there has been work and suggestion for a general purpose, compiler-blessed pointer-aliases and casting mechanism. We will let those designs take their course and instead focus on the user-facing, actionable portion of this code: dealing with `char8_t` and its related impact in C++.

§ 4.3. C Compatibility

Because of the nature of C and the fact that the only proposal on the table that is likely to be accepted is that it uses `unsigned char` (with a `typedef unsigned char char8_t` in the library), this code:

```
const unsigned char str[] = u8"";
```

may become the lingua-franca of dealing with UTF-8 in a way that is type-level different from normal non-prefixed string literals. This code will work before and after the changes proposed in [n2653]. But, it breaks when transitioning to C++20-and-beyond in headers. This can become a problem for end-users, which is why we present this as a fix. The functions in [n2730] are also going in this direction, with both papers having general approval from WG14 and slated to make it either in late C23 or early C2Y/C3Y.

Additionally, Tom Honermann's accepted `char8_t` paper and the remediation paper both state that we do not want to make it easy to convert from `u8""` and `u8'x'` literals to `char`, as that would contribute to the persistent problems on C and C++ implementations. But, there has been no harm both historically and presently to use `unsigned char` as a migration technique. Furthermore, Tom Honermann has stated that while he may not have a preference, compatibility with C is a high-order priority bit, and therefore is willing to relax his stance on that to aid in making sure C and C++ code for array initialization using `u8""` string literals continues to work.

Therefore, we additionally propose to allow initialization of `char` and `signed char` arrays. This is ultimately for parity with C code, and because `char8_t` is mandated to be exactly `unsigned char` in its underlying type in C++ this is a

completely harmless change. It is also okay to allow it, since it is an entirely deliberate action (initialization) and not anything more nefarious (like implicit conversion to a different pointer type).

We do not propose allowing `const char8_t xx[] = "text"` ("up-scaling" from normal, `char[]` string literals to `char8_t/unsigned char` literals). Even though C allows this as a natural consequence of its more-lax initialization rules, we do not allow this in C++ specifically to prevent mixing locale-based data. At the very least, someone should need to annotate their string literal with a `u8` prefix. Even if we are adding new forms of deliberate initialization, all of the initializations we are adding either fully preserve or provide a safe degradation. UTF-8 data within a locale-associated `char` type **can** be valid; locale data into a UTF-8 type is far more risky and implementation-dependent.

§ 4.4. Defect Report

This paper is being pushed forward as a Defect Report to C++20, which is when `char8_t` was first introduced. The goal is to make sure that we do not preserve an arbitrarily difficult compatibility pain. It does not truly matter what standard it is integrated into the C++ Standard, so long as implementations understand it's a defect report and should be migrated back to C++20.

§ 4.5. What about special `unsigned char*` rules?

We do not propose `unsigned char*` as allowed to be initialized with a `u8""` string literal. This is strictly due to rules around `constexpr` and current implementation limits. Forming a pointer to a block of storage which is not officially of the same type can be mocked up in the frontend, but most `constexpr` interpreters in compilers break when actually accessing the values, stating that it is not actually of the correct type (or just SEGFAULT-ing/Internal Compiler Error (ICE)-ing). This is simply a consequence of having a `char8_t` type versus just using `unsigned char`. This problem would also persist even before C++20, where `char` storage cannot be accessed with an `const unsigned char*` pointer in `constexpr` engines, even if one manages to use faux-launders techniques (as the author has experimented with in the Clang and GCC frontends). Note this is not a permanent limitation: special recognition for initialization an `const unsigned char*` from a `u8""` string literal can change it so that the backing storage for the `const unsigned char*` is of the right type.

Still, this problem can be solved, in general, by using special `alias_cast(...)` special rules or similar. But that should be a separate proposal: this proposal provides a safe, `constexpr`-friendly way to access string storage by simply first storing it in an array. This is not the most ergonomic and does not help when passed **directly** to functions rather than first stored in a `const unsigned char[]` first. It is unfortunate, but that is the price of WG14 and WG21 ignoring the few folks who called out that a `char8_t` type was needed in the earlier days. The paper that standardized `char16_t` and `char32_t` explicitly stated that they simply believed that `char` and locale work was enough, as did WG14's papers on this subject also concluded.

Clearly, this was not the case and has continued to be an enduring problem, but there is little we can do now to solve this problem besides accept that we made a mistake in C++11 and try to course correct sooner, rather than later.

§ 4.5.1. Compound Literals with C?

One way to get a `const unsigned char*` is to use C's compound literal syntax:

```
void f(const unsigned char*);
f((unsigned char[]){u8"text"});
```

This is overtly verbose and, unfortunately, compound literals are not supported in Standard C++ (though they are supported as an implementation extension in some C++ compilers with C modes, such as Clang). There is a proposal for compound literals that has seen some renewed interest over the last year, Zhihao Yuan's [\[p2174r0\]](#). It has not progressed but has been brought up for multiple use cases, meaning that it may once more be brought forward. This can be seen as an alternative solution that can be made viable by Yuan's proposal, but is not pursued in this one.

§ 4.5.2. But you CAN make it work??

In a way, yes, but it would get messy to solve this for all existing use cases. For example, consider the following code (using C++20 with all of its features available):

```
#include <cstdio>

void f(const unsigned char* f) {
    printf("%s", "unsigned char\n");
}

void f(const char* f) {
    printf("%s", "char\n");
}

void f(const char8_t* f) {
    printf("%s", "char8_t\n");
}

int main () {
    // (1)
    const unsigned char* p = u8"";
    // (2)
    f(u8 "");
    return 0;
}
```

The case for the code under `// (1)` is clear and unambiguous. One could easily argue that rather than the compiler creating a `const char8_t[]` magical static storage duration array, the initialization tells the compiler to change that and instead create a `const unsigned char[]` magic static storage duration array instead. That would allow that code to work unambiguously in C and C++. However, strictly speaking, not even the C standard blesses `// (1)`:

```
<source>:17:34: error: pointer targets in initialization
of 'const unsigned char *' from 'char *'
differ in signedness [-Werror=pointer-sign]

17 |         const unsigned char* p = u8"";
```

(Uses `-std=c2x -O3 -Wall -Wpedantic -Werror` on any Clang/GCC compiler.)

This makes the case for `// (2)` less legitimate. The only cross-platform way before C++20 to initialize something related to `unsigned char` from a string literal was an (optionally brace-enclosed) initialization for an array, `unsigned char[]`. While it would be "nice" to make the function call `f(u8 "")` immediately pick `unsigned char*` for C++, it would be wrong to add such a special exemption to C++ and then have to port that same exemption into C. This problem also does not exist for C after [n2653]: while Clang has an attribute for overloading, C does not support overloading. It will call a normal `void f(unsigned char* s)`, non-overloaded function without warning or error after [n2653]. It will also call it before the inclusion of [n2653] under normal implementation conditions (e.g., no `-Werror/-Wpedantic/-Wall/-W4`/etc.).

Thusly, we consider only the array initialization case, since this paper primarily focuses on compatibility. We also do not want to disturb overload sets which contain a choice between `void process(unsigned char*)` and `void process(char*)`, where one expects binary data and the other expects "text" (in whatever encoding). While `std::byte` can be used to break the tie, that is a newer feature and not one we can rely on safely covering the majority of C++ code out in the wild. Backwards compatibility is a goal here, and this paper is meant to make it easier, not harder.

We do think that, in the future, there can be improved interoperability with `const char*` and `const unsigned char*`. But, that will involve a great deal of additional effort, especially when it comes to how `u8 ""` may decay into a `const char*` or `const unsigned char*`, what the ranking is for overloading, and when/where it applies. This should be addressed in a future paper.

§ 4.6. Overload Resolution for Array-Containing Structure Initialization

There exists an ambiguity when initializing character arrays from `char` and, after this paper, `char8_t` literals.

The question of whether or not this matters, in overall analysis, leans into it not having significant impact. This same kind of code snippet has similar impact for string literal initialization using a plain `char`, where `unsigned char` and `signed char` usage can clash with a plain `char` array using brace initialization:

```
struct A {
    unsigned char s[10];
};
struct B {
    char s[10];
};

void f(A);
void f(B);

int main() {
    f({""}); // ambiguous
}
```

This situation now becomes the same deal when workign with `u8""` in this scenario and having `char8_t` as the aggregate initialization:

```
struct C {
    char8_t s[10];
};
struct D {
    char s[10];
};

void f(C);
void f(D);

int main() {
    f({u8""}); // ambiguous
}
```

Users could not rely on this code successfully disambiguating before C++20, going back to it being ambiguous for this very specific case is fine. Furthermore, this only applies in C++ with C-like aggregate structures: C has no such problem in its codebases, and so it should not show up at all in C code being ported to C++. Because this paper is a Defect Report, it restores it to the behavior it's had since C++11, meaning that there has been very little time for this to manifest. Given that there has been a lack of `char8_t` support in the standard library and that C has no distinct `char8_t` type (it still produces an array of `char`, albeit that might change as pointed at by previously-mentioned papers for the C Committee), this is even less likely to be a problem.

§ 5. Specification

The specification is relative to the latest C++ Working Draft, [\[n4901\]](#).

§ 5.1. Language Wording

§ 5.1.1. Adjust Feature Test Macro for `char8_t` in [\[tab:cpp.predefined.ft\]](#)

Editor's Note: Please replace with a suitable value.

Macro Name	Value
<code>__cpp_char8_t</code>	201811L 202XXXL

§ 5.1.2. Modify Initialization of Character Arrays in [dcl.init.string]

9.4.3 Character arrays [dcl.init.string]

- 1 An array of ordinary character type ([basic.fundamental]), `char8_t` array, `char16_t` array, `char32_t` array, or `wchar_t` array **can** **may** be initialized by an ordinary string literal, UTF-8 string literal, UTF-16 string literal, UTF-32 string literal, or wide string literal, respectively, or by an appropriately-typed *string-literal* enclosed in braces ([lex.string]). **Additionally, an array of `char` or `unsigned char` may be initialized by a UTF-8 string literal, or by such a string literal enclosed in braces.** Successive characters of the value of the *string-literal* initialize the elements of the array, **with an integral conversion [conv.integral] if necessary for the source and destination value.**

§ 5.1.3. Add Annex C.1.6 example for change in code [diff.cpp20]

C.1.6 [dcl.dcl]: Declarations [diff.cpp20.dcl]

- 1 **Affected subclaus:** [dcl.init.string]

Change: UTF-8 string literals may initialize arrays of ordinary character type, instead of only `char8_t` type.

Rationale: Compatibility with previously written code that conformed to previous versions of this document.

Effect on original feature: Arrays of ordinary character type may now be initialized with a UTF-8 string literal. This can affect initialization that includes arrays which are directly initialized within class types, typically aggregates.

[Example 1:

```
struct A {
    char8_t s[10];
};
struct B {
    char s[10];
};

void f(A);
void f(B);

int main()._{
    f({u8""}); // ambiguous
}
```

— end example]

§ References

§ Informative References

[MICROSOFT-UTF8]

Microsoft. . June 24th, 2021. URL: <https://docs.microsoft.com/en-us/windows/apps/design/globalizing/use-utf8-code-page>

[N1985]

Jack W. Reeves. [Request the Standard Provide Explicit Specialization of `char\_traits` For All Built-in Character Types](#). 6 April 2006. URL: <https://wg21.link/n1985>

[N2271]

Paul Pedriana. [EASTL -- Electronic Arts Standard Template Library](#). 27 April 2007. URL: <https://wg21.link/n2271>

[N2653]

Tom Honermann. *char8_t: A type for UTF-8 characters and strings*. June 4th, 2021. URL: <http://www.open-std.org/jtc1/sc22/wg14/www/docs/n2653.htm>

[N2730]

JeanHeyd Meneide; Shepherd. *Restartable and Non-Restartable Functions for Efficient Character Conversions*. November 30th, 2021. URL: <http://www.open-std.org/jtc1/sc22/wg14/www/docs/n2730.htm>

[N4901]

Thomas Köppe. *Working Draft, Standard for Programming Language C++*. October 23rd, 2021. URL: <https://wg21.link/n4901>

[P1423r3]

Tom Honermann. *char8_t backward compatibility remediation*. 20 July 2019. URL: <https://wg21.link/p1423r3>

[P2174R0]

Zhihao Yuan. *Compound Literals*. 16 May 2020. URL: <https://wg21.link/p2174r0>